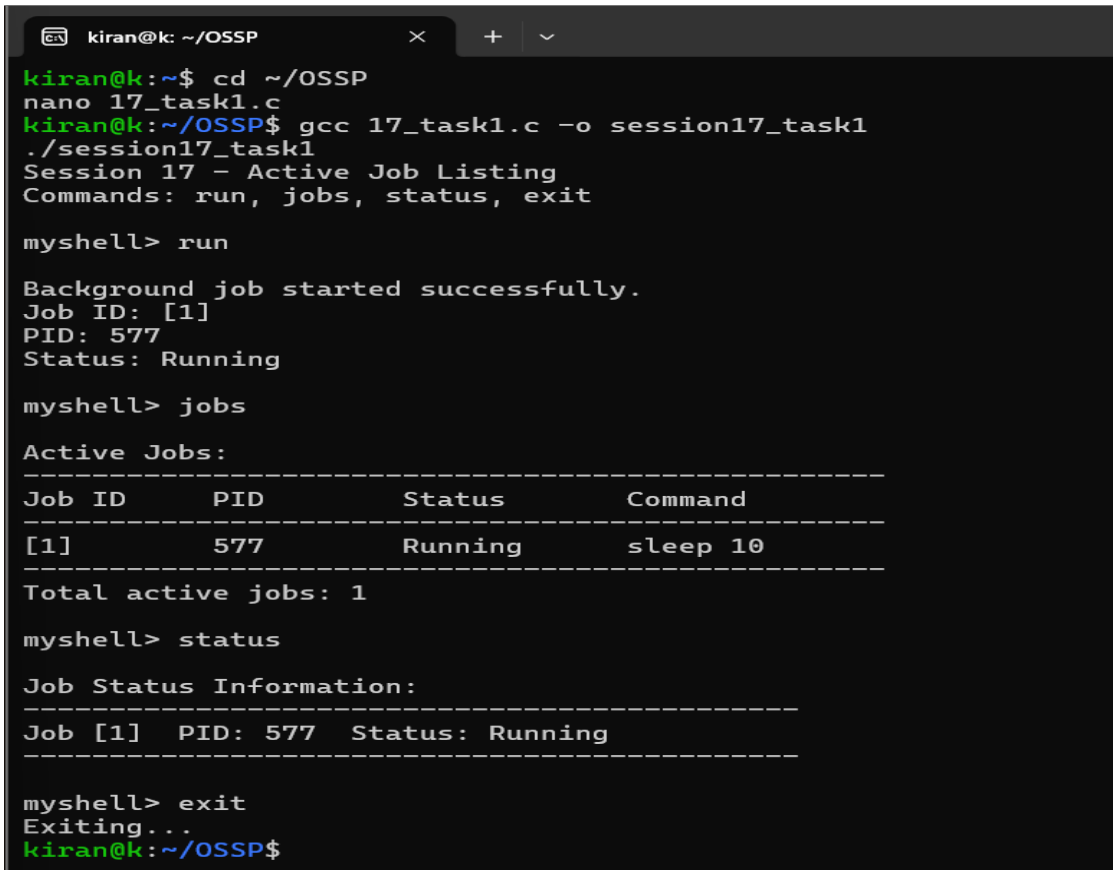
Skill Week-9

Session-17

Task 1: To implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.



```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 10

struct Job
{
    int job_id;
    pid_t pid;
    char state[20];
};

struct Job jobs[MAX_JOBS];
int job_count = 0;

void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        int status;
        pid_t result = waitpid(jobs[i].pid, &status,
                               WNOHANG | WUNTRACED | WCONTINUED);

        if (result > 0)
        {
            if (WIFEXITED(status))
                snprintf(jobs[i].state, 20, "Done");
            else if (WIFSTOPPED(status))
                snprintf(jobs[i].state, 20, "Stopped");
            else if (WIFCONTINUED(status))
                snprintf(jobs[i].state, 20, "Running");
        }
    }
}

void list_jobs()
{
    update_jobs();

    printf("\nJob ID\tPID\tState\n");
    printf("-------------------------\n");

    for (int i = 0; i < job_count; i++)
    {
        printf("[%d]\t%d\t%s\n",
               jobs[i].job_id,
               jobs[i].pid,
               jobs[i].state);
    }
}
```

```c
int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process started. PID = %d\n", getpid());

        sleep(10);

        printf("Child process completed.\n");
        exit(0);
    }

    jobs[job_count].job_id = 1;
    jobs[job_count].pid = pid;
    snprintf(jobs[job_count].state, 20, "Running");
    job_count++;

    printf("Created Job 1 with PID %d\n", pid);

    list_jobs();

    sleep(3);

    list_jobs();

    waitpid(pid, NULL, 0);

    snprintf(jobs[0].state, 20, "Done");

    list_jobs();

    return 0;
}
```

Task 2: To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

```
kiran@k:~/OSSP$ cd ~/OSSP
nano 18_task1.c
kiran@k:~/OSSP$ cd ~/OSSP
nano 17_task2.c
kiran@k:~/OSSP$ gcc 17_task2.c -o foreground_job
./foreground_job
Parent: Terminal transferred to child.
Child: PID = 1257
Child: Running as foreground job...
Child: Working... 1/5
Child: Working... 2/5
Child: Working... 3/5
Child: Working... 4/5
Child: Working... 5/5
Child: Completed.

Parent: Terminal control returned.
Foreground job completed successfully.
Job state updated: Completed.
kiran@k:~/OSSP$
```

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        // Create a new process group
        setpgid(0, 0);

        printf("Child: PID = %d\n", getpid());
        printf("Child: Running as foreground job...\n");

        for (int i = 1; i <= 5; i++)
        {
            printf("Child: Working... %d/5\n", i);
            sleep(1);
        }

        printf("Child: Completed.\n");
        exit(0);
    }

    // Parent places child in its own process group
    setpgid(pid, pid);

    // Give terminal control to child process group
    tcsetpgrp(STDIN_FILENO, pid);

    printf("Parent: Terminal transferred to child.\n");

    // Wait for child
    waitpid(pid, &status, WUNTRACED);

    // Take terminal control back
    tcsetpgrp(STDIN_FILENO, getpgrp());

    if (WIFEXITED(status))
    {
        printf("Parent: Child completed normally.\n");
    }
    else if (WIFSTOPPED(status))
    {
        printf("Parent: Child was stopped.\n");
    }

    printf("Parent: Terminal control restored.\n");

    return 0;
}
```

Session-18

Task 1: To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

```
kiran@k:~/OSSP$ nano 18_task1.c
kiran@k:~/OSSP$ gcc 18_task1.c -o signal_demo -Wall -Wextra
kiran@k:~/OSSP$ ./signal_demo
Child 1: PID = 1537, PGID = 1536
Child 1: Blocking SIGUSR1 for 3 seconds.
Child 2: PID = 1538, PGID = 1536

Parent: Sending SIGUSR1 to process group 1536
Child received signal 10
Child received signal 10
Child 1: Unblocking SIGUSR1.
Child received signal 10
Parent: Both children completed.
kiran@k:~/OSSP$
```

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void signal_handler(int sig)
{
    printf("Child received signal %d\n", sig);
}

int main()
{
    pid_t child1, child2;
    pid_t pgid;

    child1 = fork();

    if (child1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (child1 == 0)
    {
        signal(SIGUSR1, signal_handler);

        setpgid(0, 0);

        printf("Child 1: PID = %d, PGID = %d\n",
               getpid(), getpgrp());

        printf("Child 1: Blocking SIGUSR1 for 3 seconds.\n");

        sigset_t mask;

        sigemptyset(&mask);
        sigaddset(&mask, SIGUSR1);

        sigprocmask(SIG_BLOCK, &mask, NULL);

        sleep(3);

        printf("Child 1: Unblocking SIGUSR1.\n");

        sigprocmask(SIG_UNBLOCK, &mask, NULL);

        sleep(2);

        exit(0);
    }
```

```c
    child2 = fork();

    if (child2 < 0)
    {
        perror("fork");
        return 1;
    }

    if (child2 == 0)
    {
        signal(SIGUSR1, signal_handler);

        setpgid(0, child1);

        printf("Child 2: PID = %d, PGID = %d\n",
               getpid(), getpgrp());

        sleep(10);

        exit(0);
    }

    // Parent makes both children part of same group
    pgid = child1;

    setpgid(child1, pgid);
    setpgid(child2, pgid);

    sleep(1);

    printf("\nParent: Sending SIGUSR1 to process group %d\n",
           pgid);

    kill(-pgid, SIGUSR1);

    waitpid(child1, NULL, 0);
    waitpid(child2, NULL, 0);

    printf("Parent: Both children completed.\n");

    return 0;
}
```

Task 2: To implement how to resume stopped jobs, send continue signals, update job status, maintain job table, verify execution, test background recovery.

```
kiran@k:~/OSSP$ cd ~/OSSP
kiran@k:~/OSSP$ nano resume_job.c
kiran@k:~/OSSP$ gcc resume_job.c -o resume_job -Wall -Wextra
kiran@k:~/OSSP$ ./resume_job
Child: PID = 1609
Child: Running and will stop itself.
Child: Sending SIGSTOP to itself.
Parent: Child is stopped.
Parent: Sending SIGCONT.
Child: Resumed after SIGCONT.
Child: Working 1/5
Child: Working 2/5
Child: Working 3/5
Child: Working 4/5
Child: Working 5/5
Child: Finished.
Parent: Child completed successfully.
kiran@k:~/OSSP$
```

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child: PID = %d\n", getpid());
        printf("Child: Running and will stop itself.\n");

        sleep(2);

        printf("Child: Sending SIGSTOP to itself.\n");

        raise(SIGSTOP);

        // Execution resumes here after SIGCONT
        printf("Child: Resumed after SIGCONT.\n");

        for (int i = 1; i <= 5; i++)
        {
            printf("Child: Working %d/5\n", i);
            sleep(1);
        }

        printf("Child: Finished.\n");

        exit(0);
    }

    // Wait until child stops
    waitpid(pid, &status, WUNTRACED);

    if (WIFSTOPPED(status))
    {
        printf("Parent: Child is stopped.\n");
        printf("Parent: Sending SIGCONT.\n");

        kill(pid, SIGCONT);
    }

    // Wait until child finishes
    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("Parent: Child completed successfully.\n");
    }

    return 0;
```